기존의 AutoML 시스템을 분석하고, 기능을 장점만 가려내서 GPU 기반의 AutoML 프레임워크를 오픈소스로 만드는 것이 핵심 목표

* stacked ensemble 방법론 및 원리

* hyper parameter tuning 방법론 및 원리

=> 위 두 가지를 조사해서 어떻게 RAPIDS를 기반으로 하는 End-to-End GPU based AutoML System에 적용할 수 있는지 조사

* Stacked ensemble

- **개념**

Stacked Ensemble(스택 앙상블)은 여러 개의 모델(Base model)의 예측 결과를 입력으로 사용하여, 이를 결합하는 새로운 모델(Meta model)을 학습하는 기법이다.

구조는 다음과 같다.

[Base Models] --- GBM / XGBoost / RF / DNN

모델1 → 예측1

모델2 → 예측2

모델3 → 예측3

↓

[Meta Model] --- GLM
예측1, 예측2, 예측3을 입력으로 다시 학습
↓

최종 예측 생성

- **성능 향상 원리**

각 모델은 서로 다른 방식으로 오류를 발생시키기 때문에,
여러 모델을 결합하면 개별 모델의 약점을 보완할 수 있다.

- **데이터 누수 방지 (핵심)**

Stacking에서 가장 중요한 문제는 **데이터 누수(Data Leakage)**이다.

- 같은 데이터로 학습하고 같은 데이터로 예측하면
  → 과적합 발생

이를 방지하기 위해:

Cross Validation 기반 Out-of-Fold 예측을 사용

- **H2O의 Stacking 전략**

H2O AutoML은 Stacking을 핵심 기능으로 사용하며, 특히 다음 두 가지 앙상블을 생성한다.

① All Models Ensemble

- 가능한 많은 모델을 포함
- 최고 성능 목표

② Best of Family Ensemble

- 알고리즘별 최적 모델만 선택
- 예측 속도 향상

-> 이는 "성능 vs 속도" 트레이드오프를 동시에 해결하는 설계이다.

- **Meta Model 설계**

H2O는 기본적으로 다음을 사용한다.

GLM (선형 모델, 비음수 가중치)

이 선택의 장점: 안정성, 해석 가능성, 빠른 연산

- **GPU 환경에서의 Stacking**

Stacking은 GPU 환경에서 다음과 같은 문제를 유발한다.

○ **메모리 문제**

Stacking 시 생성되는 데이터:

샘플 수 × 모델 수

예:

- 1,000,000 샘플 × 100 모델 → 매우 큰 행렬

-> GPU 메모리 부족 가능

○ **해결 전략**

GPU 기반 AutoML에서는 다음이 필요하다.

① GPU 메모리 유지 (Zero-copy 전략)

- CPU ↔ GPU 이동 최소화

② 분산 처리 / 스트리밍

- 전체 데이터를 한 번에 처리하지 않음

③ 모델 수 제한

- Best of Family 전략 활용

---

* Hyper parameter tuning

- **개념**

Hyperparameter는 모델의 구조와 학습 방식을 결정하는 설정값이다.

예: learning rate, tree depth, number of trees

- **H2O의 탐색 방식**

H2O는 다음 두 가지 방식을 지원한다.

① Grid Search

- 모든 조합 탐색
- 비용 큼

② Random Search (핵심)

- 랜덤 샘플링
- 효율적 탐색

-> 실제로 Random Search만으로도 충분히 높은 성능 확보 가능

- **AutoML 내부 구조**

H2O AutoML은 다음 단계로 구성된다.

1. 기본 모델 생성
2. Random Search
3. (선택) Fine-tuning
4. Ensemble 구성

- **시간 기반 제어**

AutoML은 다음 기준으로 실행된다.

- max_runtime_secs, max_models

-> 즉: "주어진 시간 안에서 최대 성능 달성"

- **알고리즘별 시간 분배**

H2O는 알고리즘별로 중요도를 다르게 설정한다.

예:

- XGBoost → 더 많은 시간
- 기타 모델 → 상대적으로 적은 시간

-> 이는 자원 효율성을 높이는 핵심 전략이다.

- **GPU 환경에서의 HPO**

○ **장점**

GPU에서는 다음이 가능하다.

- 여러 모델을 동시에 학습 (병렬화)
- 탐색 속도 대폭 증가

○ **문제점**

① GPU 메모리 제한

- 동시에 너무 많은 모델 실행 시 OOM 발생

② 재현성 문제

- GPU 연산 특성상 결과 변동 가능

○ **해결 전략**

✓ Resource-aware scheduling

- GPU 메모리 고려한 실행

✓ Deterministic 모드

- 재현성 확보

- **GPU 기반 AutoML 설계 전략**

○ **데이터 계층**

가장 중요한 원칙: 데이터는 GPU에서 상주할 것.

- cuDF 기반 DataFrame 활용
- 컬럼형 데이터 구조 유지

○ **모델 및 앙상블 계층**

H2O 구조를 그대로 적용:

- All Models (성능)
- Best of Family (속도)

Meta model:

- 단순 선형 모델 유지

○ **HPO 계층**

핵심 설계 방향:

- Random Search 중심
- Trial 병렬화
- 자원 기반 스케줄링

- **GPU 기반 AutoML 통합 전체 아키텍처**

데이터를 GPU에 상주시킨 상태에서, fold-aware 전처리와 resource-aware HPO 스케줄링을 통해 base 모델들을 병렬 학습하고, OOF 예측으로 All Models / Best of Family 스택 앙상블을 만든 뒤, 정확도-지연시간 기준으로 최종 배포 모델을 선택하는 구조

[GPU Data Layer]
- cuDF 기반 데이터 적재 및 GPU 상주

↓

[Feature Engineering]
- GPU 전처리 및 CV fold 준비

↓

[HPO Scheduler]
- Random Search
- Trial 병렬화
- 자원 기반 스케줄링

↓

[Parallel Model Training]
- XGBoost / cuML 기반 GPU 학습
- OOF 예측 생성

↓

[Stacked Ensemble]
- All Models
- Best of Family
- Meta Learner

↓

[Model Selection]
- Best Accuracy
- Best Latency
- 최종 배포 모델 선택